

# ClaimShield

An internal claim management system for health insurance administrators

|                       |                                                                                                                                                                                               |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Name</b>           | Nguyen Ba Nam                                                                                                                                                                                 |
| <b>Student number</b> | S3974998                                                                                                                                                                                      |
| <b>Lecturer</b>       | Minh Vu                                                                                                                                                                                       |
| <b>GitHub link</b>    | <a href="https://github.com/RMIT-Vietnam-Teaching/java-console-application-part-1-NguyenBaNam109">https://github.com/RMIT-Vietnam-Teaching/java-console-application-part-1-NguyenBaNam109</a> |

---

## 1. Application Description

### 1.1 Purpose and scope

ClaimShield is a text based internal system that lets a health insurance administrator keep track of three linked kinds of record: the customers covered by the company, the insurance cards issued to those customers, and the claims filed against a medical event. Milestone 1 delivers the foundation of that system: a working in-memory data layer, full create, read, update and delete support for all three record types, a validated console interface, and text file persistence that survives a restart.

The scope is deliberately limited to what the milestone asks for. The program uses plain classes, ArrayList collections and simple file input and output. Inheritance, abstract classes and interfaces are not used, because with only three unrelated entity types they would add indirection without removing any duplication. Section 4 explains where those features become worthwhile in Milestone 2.

### 1.2 Architecture and package structure

The program is split into five packages, each with one responsibility. The dependencies point in one direction only: the console talks to the manager, the manager owns the entities and consults the utilities, and the persistence package is the only code that knows a file exists. No entity class imports the console, and no console class parses a data line.

| Package             | Classes                        | Responsibility                                                                                    |
|---------------------|--------------------------------|---------------------------------------------------------------------------------------------------|
| claimshield         | ClaimShieldApp                 | Entry point. Builds the layers, loads the data files, starts the menu.                            |
| claimshield.model   | Customer, InsuranceCard, Claim | Entity state only. Private fields, getters and setters, no persistence and no cross-record logic. |
| claimshield.manager | ClaimManager                   | The in-memory database. Owns the three ArrayLists and enforces every business rule.               |
| claimshield.io      | FileStorage, RecordMapper      | Reads and writes the data files, and converts between a text line and an object.                  |
| claimshield.util    | Validator, IdGenerator         | Reusable format rules and automatic id generation.                                                |

| Package        | Classes                  | Responsibility                                      |
|----------------|--------------------------|-----------------------------------------------------|
| claimshield.ui | ConsoleMenu, InputReader | Screens, prompts, re-prompting and error reporting. |

This separation is what keeps the code short. Because every rule lives in ClaimManager, a record typed by an administrator and a record read from a data file are judged by exactly the same code, so the program cannot end up with data on disk that it would have refused at the keyboard.

### 1.3 Entity model and relationships

A Customer is either a PolicyHolder, who pays for their own plan and stores null in parentPolicyHolderId, or a Dependent, who must point at an existing PolicyHolder. An InsuranceCard links the customer who carries it to the policy holder paying for it, and a Claim points at the insured customer and at the card used for the medical event. All three links are stored as string ids rather than object references, which keeps the entity classes independent of each other and makes the flat text file format a direct mirror of the object model.

| Entity        | Key                    | Points at                      | Notes                                                                   |
|---------------|------------------------|--------------------------------|-------------------------------------------------------------------------|
| Customer      | id (c-7digits)         | parentPolicyHolderId           | null for a policy holder, an existing policy holder id for a dependent. |
| InsuranceCard | cardNumber (10 digits) | cardHolderId,<br>policyOwnerId | The payer is the holder themselves, or the holder's policy holder.      |
| Claim         | id (f-10digits)        | insuredPersonId,<br>cardNumber | Holds its own list of document file names and its workflow status.      |

### 1.4 Business rules enforced

Every rule below is checked inside ClaimManager. When a rule is broken the operation throws an IllegalArgumentException carrying a message written for an administrator; the console prints that message and nothing in memory changes, and the file loader turns the same message into a skipped-line warning.

| Rule                                                                                                        | Where it is enforced                                   |
|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Customer ids match c- plus 7 digits; claim ids match f- plus 10 digits; card numbers are exactly 10 digits. | Validator, called from every add operation             |
| Customer ids, claim ids and card numbers are unique across the system.                                      | ClaimManager.addCustomer / addClaim / addInsuranceCard |
| A claim amount must be greater than zero.                                                                   | Validator.isValidClaimAmount                           |
| A free text value such as the full name must not contain the   field separator.                             | Validator.isStorableText                               |
| The exam date must fall before, or on the same calendar day as, the claim date.                             | Validator.isExamBeforeOrSameDayAsClaim                 |
| The exam date must fall before the expiry of the card used.                                                 | Validator.isExamBeforeCardExpiry                       |
| A document must be named ClaimId_CardNumber_DocumentName.pdf.                                               | Validator.isValidDocumentName                          |
| A status may only move one step forward: New to Processing, then Processing to Done.                        | Validator.isForwardStatusTransition                    |

| Rule                                                                                         | Where it is enforced                              |
|----------------------------------------------------------------------------------------------|---------------------------------------------------|
| A dependent must reference an existing policy holder; a policy holder must reference nobody. | ClaimManager.validateParentReference              |
| A card must be paid for by the holder, or by the holder's policy holder.                     | ClaimManager.validateCardOwnership                |
| A claim may only use a card that belongs to the insured person.                              | ClaimManager.addClaim                             |
| A record still referenced by another record cannot be deleted.                               | ClaimManager.deleteCustomer / deleteInsuranceCard |

## 1.5 Data file format and persistence

The system stores its data in three plain text files inside the data folder, one record per line. Fields are separated by a vertical bar rather than a comma, because a customer name such as "Nguyen Thi Mai, Jr" would otherwise split a record in two. Documents inside a claim are joined with a semicolon, the literal null marks an absent policy holder, and all dates are written as ISO-8601 text so the files stay readable and can be parsed straight back into LocalDateTime. The first line of each file is a comment describing the layout, and any line starting with # is ignored on load.

```
customers.txt      id|fullName|customerType|parentPolicyHolderId
c-1000001|Nguyen Van An|PolicyHolder|null
c-1000011|Nguyen Bao Long|Dependent|c-1000001

insurance_cards.txt  cardNumber|cardHolderId|policyOwnerId|expirationDate
1000000001|c-1000001|c-1000001|2027-12-31T23:59:00

claims.txt
id|claimDate|insuredPersonId|cardNumber|examDate|documents|claimAmount|status
f-1000000001|2026-01-13T10:00:00|c-1000001|1000000001|2026-01-12T09:30:00|f-
1000000001_1000000001_Invoice.pdf;f-1000000001_1000000001_Prescription.pdf|1850.0|Done
```

The three files ship with 20 sample records each, well above the 15 required, and every cross reference between them is consistent. Loading happens in dependency order, customers first, then cards, then claims, because a card needs its customer and a claim needs both its customer and its card. Loading is fault tolerant: a malformed or rule-breaking line is reported with its file name, line number and reason, then skipped, so the program always starts and the administrator can see exactly what to repair. Choosing Save and Exit writes the whole in-memory state back over the three files. The save itself is protected: each file is first written to a temporary neighbour, and the three real files are swapped in only after all of them were written successfully, so an interrupted save cannot leave a half written file behind.

## 1.6 Console interface

The main menu offers the four required options and shows a live count of the records in memory. Each of the first three options opens a sub menu with option 0 always mapped to Back, and typing the word cancel at any prompt aborts the current operation without changing anything. Numeric prompts re-prompt instead of crashing, ids are checked against their pattern before the manager is called, and both id fields and card numbers can be auto-generated so the administrator does not have to invent a unique value. Results are reported with a short [OK] or [!] line.

```
-----
CLAIMSHIELD - Health Insurance Administration Console
20 customers | 20 insurance cards | 20 claims in memory
-----
1. Manage Customer Directory
```

2. Manage Insurance Cards
3. Process Claims
4. Save and Exit

-----  
Select an option (1-4):

## 1.7 Software engineering practices applied

Rather than name-dropping patterns, the code base applies a small set of practices consistently.

- Encapsulation. Every field is private, and every collection that leaves a class - the three lists in ClaimManager and the document list in Claim - is exposed only as an unmodifiable view, so the sole way to change data is through a method that validates it first.
- Single responsibility. Entities hold state, ClaimManager holds the rules, FileStorage holds the files and ConsoleMenu holds the screens, so changing the file format or the menu wording touches exactly one package.
- Fail fast with actionable messages. An operation that cannot complete throws immediately, nothing is half applied, and the message tells the administrator precisely what to fix.
- One source of truth for every rule. Each check exists exactly once, in Validator or ClaimManager, so the console and the file loader can never disagree about what a valid record is.
- Consistent style. Standard Java naming, four space indentation, Javadoc on every public member, and a build that is clean under javac -Xlint:all.

## 1.8 Testing performed

The program was compiled with javac -Xlint:all and produces no errors and no warnings. It was then driven through scripted console sessions covering the happy path and each rule. The table below lists the checks that were run and what the program printed.

| Scenario exercised                                          | Observed result                                                          |
|-------------------------------------------------------------|--------------------------------------------------------------------------|
| Start with the shipped data files                           | 60 records loaded: 20 customers, 20 cards, 20 claims                     |
| Save and Exit, then compare the files with the originals    | Byte-for-byte identical, so the round trip loses nothing                 |
| Add a customer, register a card, file a claim, then restart | All three new records reloaded correctly                                 |
| Delete a policy holder who still has dependents             | Refused with an explanation, nothing changed                             |
| Register a second card for a customer who already holds one | Refused                                                                  |
| File a claim whose exam date is after the claim date        | Refused                                                                  |
| Move a claim straight from New to Done                      | Refused, one step forward only                                           |
| Move a claim from Done back to New                          | Refused                                                                  |
| Attach a document with a wrong file name                    | Refused with the exact required file name shown                          |
| Type a customer name containing the   character             | Refused, because   separates the fields of the data files                |
| Turn a card-holding policy holder into a dependent          | Refused until the card is removed, keeping the recorded payer consistent |
| Type letters and out-of-range numbers at a menu prompt      | Re-prompted, no crash                                                    |

| Scenario exercised                           | Observed result                                                          |
|----------------------------------------------|--------------------------------------------------------------------------|
| Load a data file containing a corrupted line | Line skipped with file, line number and reason;<br>program still started |

## 2. Application Flow (Diagram)

Figure 1 shows the life cycle of one run of the program, from loading the data files to writing them back. The important detail is the loop in the lower half: every user action ends in a single call to ClaimManager, which either applies the change and reports [OK] or refuses it and reports the reason, and in both cases control returns to the menu the administrator came from.

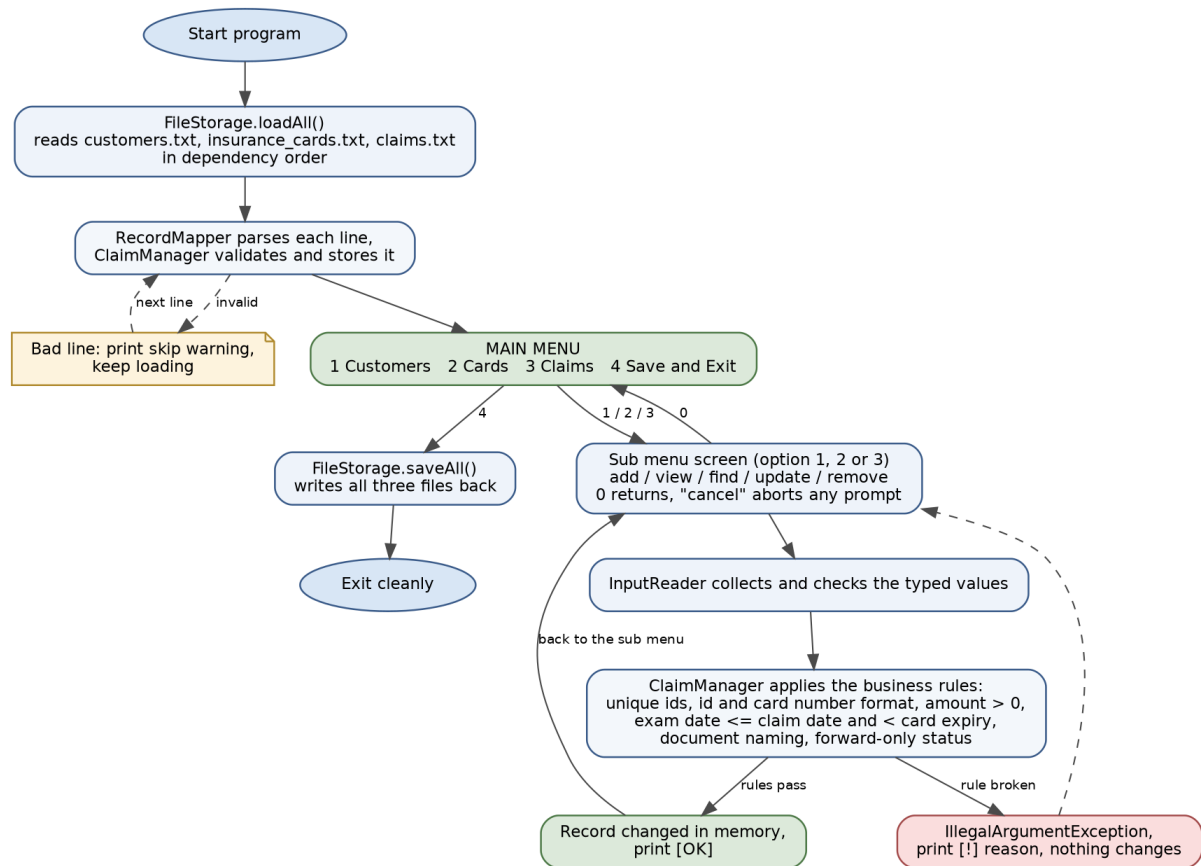


Figure 1. Program flow from startup to a clean exit.

Figure 2 shows how the packages depend on each other. Dependencies point downwards and inwards: the console layer knows the manager, the manager knows the entities and the utilities, and FileStorage is the only class that touches the disk. The dashed arrows between entities are the string id references described in section 1.3, not Java object references.

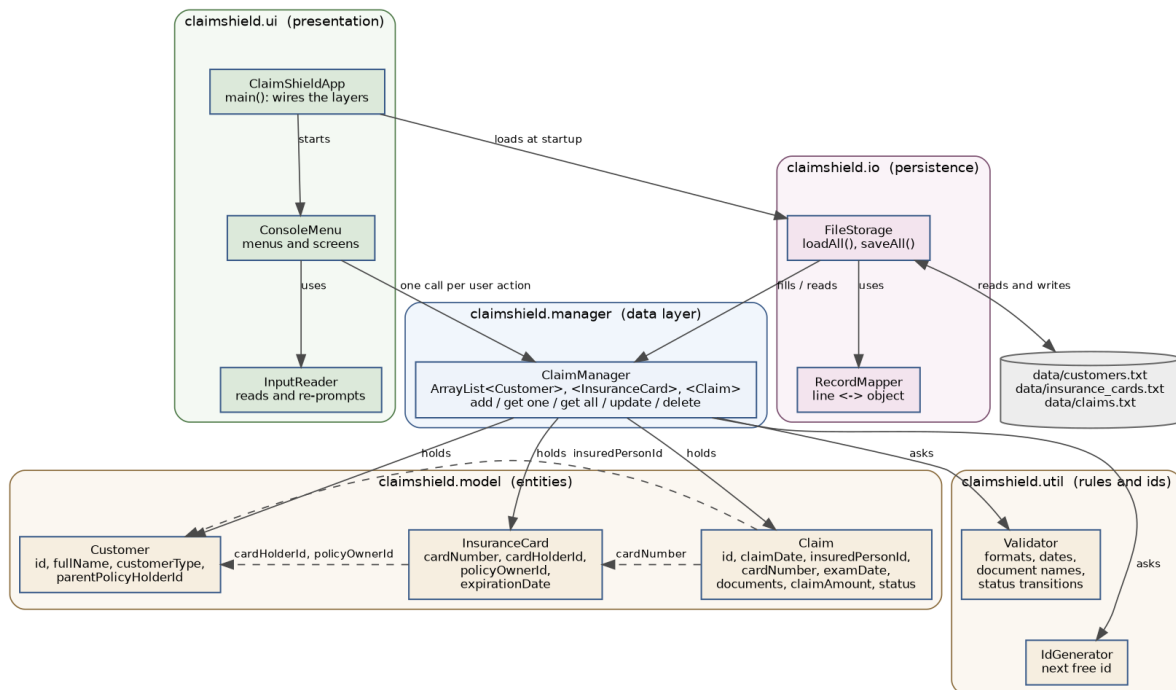


Figure 2. Package structure and the direction of dependencies.

### 3. API List

The tables below list the public methods of each class. Anything not listed is private, which is why the collections inside ClaimManager and the document list inside Claim cannot be modified from outside: getAllCustomers, getAllInsuranceCards, getAllClaims and getDocuments all return unmodifiable views, so the only way to change the data is through a method that validates it first.

#### ClaimManager - customer operations

| Method                                         | Description                                                                                          |
|------------------------------------------------|------------------------------------------------------------------------------------------------------|
| addCustomer(Customer)                          | Validates the id format, uniqueness, name, type and policy holder reference, then stores the record. |
| getCustomerById(String)                        | Returns the customer with that id, or null.                                                          |
| getAllCustomers()                              | Returns an unmodifiable view of every customer.                                                      |
| updateCustomer(String, String, String, String) | Changes the name, type and policy holder of an existing customer. The id itself is immutable.        |
| deleteCustomer(String)                         | Removes a customer only when no dependent, card or claim still refers to them.                       |
| generateCustomerId()                           | Returns the next unused id in the c-7digits family.                                                  |
| hasDependents(String)                          | Reports whether any customer names this id as their policy holder.                                   |

#### ClaimManager - insurance card operations

| Method                          | Description                                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------|
| addInsuranceCard(InsuranceCard) | Validates the number, the holder, the payer and the one-card-per-customer rule, then stores the card. |
| getCardByNumber(String)         | Returns the card with that number, or null.                                                           |

| Method                                      | Description                                                                                             |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------|
| getCardByHolderId(String)                   | Returns the card carried by a customer, or null.                                                        |
| getAllInsuranceCards()                      | Returns an unmodifiable view of every card.                                                             |
| updateCardExpiration(String, LocalDateTime) | Moves the expiry date, and refuses the change if an existing claim would then sit outside the coverage. |
| deleteInsuranceCard(String)                 | Removes a card only when no claim refers to it.                                                         |
| generateCardNumber()                        | Returns the next unused 10-digit card number.                                                           |

### ClaimManager - claim operations

| Method                             | Description                                                                                                     |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| addClaim(Claim)                    | Validates the id, amount, customer, card and both dates, then stores the claim.                                 |
| restoreClaim(Claim, List<String>)  | Used by the persistence layer to re-insert a stored claim together with the documents it already carried.       |
| getClaimById(String)               | Returns the claim with that id, or null.                                                                        |
| getAllClaims()                     | Returns an unmodifiable view of every claim.                                                                    |
| getClaimsByCustomer(String)        | Returns every claim filed for one customer.                                                                     |
| addDocumentToClaim(String, String) | Attaches a document after checking the ClaimId_CardNumber_DocumentName.pdf convention and rejecting duplicates. |
| updateClaimStatus(String, String)  | Moves a claim one step forward through New, Processing and Done.                                                |
| updateClaimAmount(String, double)  | Changes the claimed amount of an existing claim.                                                                |
| deleteClaim(String)                | Removes a claim from the system.                                                                                |
| generateClaimId()                  | Returns the next unused id in the f-10digits family.                                                            |

### Validator and IdGenerator

| Method                                                           | Description                                                                                                   |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Validator.isValidCustomerId / isValidClaimId / isValidCardNumber | Pattern checks for the three id families.                                                                     |
| Validator.isValidCustomerType / isValidStatus / isValidFullName  | Checks a value against the allowed set, or against emptiness.                                                 |
| Validator.isValidClaimAmount(double)                             | Confirms the amount is greater than zero.                                                                     |
| Validator.isExamBeforeOrSameDayAsClaim(...)                      | Confirms the examination happened no later than the day the claim was filed.                                  |
| Validator.isExamBeforeCardExpiry(...)                            | Confirms the examination happened while the card was still valid.                                             |
| Validator.isValidDocumentName(String, String, String)            | Confirms a file name follows ClaimId_CardNumber_DocumentName.pdf.                                             |
| Validator.isForwardStatusTransition(String, String)              | Confirms a status change is exactly one step forward.                                                         |
| IdGenerator.nextCustomerId / nextClaimId / nextCardNumber        | Derives the next free value from the ids currently in use, so it stays correct after a restart or a deletion. |

### FileStorage and RecordMapper

| Method                                                                    | Description                                                                                |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>FileStorage(String)</code>                                          | Creates a storage object pointing at a data folder.                                        |
| <code>loadAll(ClaimManager)</code>                                        | Reads the three files in dependency order and returns how many records were accepted.      |
| <code>saveAll(ClaimManager)</code>                                        | Writes every record in memory back over the three files and returns how many were written. |
| <code>getWarnings()</code>                                                | Returns the skipped-line messages produced by the last load.                               |
| <code>RecordMapper.toLine(Customer / InsuranceCard / Claim)</code>        | Converts an entity into its storage line.                                                  |
| <code>RecordMapper.parseCustomer / parseInsuranceCard / parseClaim</code> | Rebuilds an entity from a storage line, reporting exactly which field is malformed.        |
| <code>RecordMapper.parseClaimDocuments(String)</code>                     | Extracts the document names from a claim line.                                             |
| <code>RecordMapper.isSkippable(String)</code>                             | Reports whether a line is blank or a comment.                                              |

## Entity and console classes

| Method                                                                                                           | Description                                                                                   |
|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>Customer.isPolicyHolder()</code>                                                                           | Reports whether the customer pays for their own plan.                                         |
| <code>InsuranceCard.isValidAt(LocalDateTime)</code>                                                              | Reports whether the card is still valid at a given moment.                                    |
| <code>Claim.getDocuments()</code>                                                                                | Returns an unmodifiable view of the attached file names.                                      |
| <code>Claim.addDocument(String) / hasDocument(String)</code>                                                     | Attaches a validated file name, or reports whether one is already attached.                   |
| <code>Claim.toDetailedString()</code>                                                                            | Renders the claim with its full document list for the detail screens.                         |
| <code>ConsoleMenu.run()</code>                                                                                   | Runs the main menu loop until the administrator saves and exits.                              |
| <code>InputReader.readMenuChoice / readText / readPattern / readPositiveAmount / readDateTime / readYesNo</code> | Collect one value each, re-prompting until it is acceptable and honouring the cancel keyword. |

## 4. Drawbacks and Future Work

### 4.1 Known drawbacks

- Linear searching. Every lookup walks an ArrayList, so finding a record is  $O(n)$  and adding one is  $O(n)$  because of the uniqueness check. With a few thousand records this is invisible, but it does not scale to a real insurer's data set.
- Changes live only in memory until Save and Exit. A crash in the middle of a long session therefore loses that session's edits. Writing the files after every successful operation, or offering a manual save option in the menu, would remove that risk.
- Three near identical loading methods. loadCustomers, loadInsuranceCards and loadClaims differ only in which parse and add calls they make. Without generics or functional interfaces this repetition is hard to remove cleanly at this stage.
- Money stored as a double. Binary floating point cannot represent every decimal amount exactly, which is acceptable for a prototype but wrong for real currency.
- No concurrency and no access control. The design assumes a single administrator at a single console, and every user has full rights over every record.
- Documents are names only. The system records file names and never checks that the file exists, so a claim can reference a document that was never uploaded.

### 4.2 Future work

- Introduce the inheritance and interface layer that Milestone 2 expects: a Customer base class with PolicyHolder and Dependent subclasses would let each type carry its own behaviour instead of a customerType string, and a generic CrudRepository interface would remove the duplication in both the manager and the loader.
- Replace the ArrayLists with HashMaps keyed by id. Lookups and uniqueness checks drop to constant time and the manager code gets shorter at the same time.
- Switch to BigDecimal for money, and add claim totals and simple reporting such as the amount claimed per policy holder or per status.
- Add automated JUnit tests for Validator and ClaimManager. The rules are already isolated in pure methods, so they are ready to be tested without a console.
- Move persistence behind a small interface so the text files can later be replaced by a database or by JSON without touching the manager or the console.
- Extend the console with searching and filtering, for example listing every claim in Processing or every card expiring within a month, and eventually replace it with a JavaFX interface.